COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Advanced engine systems and simulation research

Standalone teaching module; use the workbook and separate answer key

MODULE 44 / FOUR LESSONS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Module 44 — Advanced engine systems and simulation research 3

Lesson 44.1 — Large-world scheduling, streaming and critical-path control 4

Lesson 44.2 — Deformable simulation, variational models and solver tradeoffs 6

Lesson 44.3 — Distributed game state, consistency and overload behavior 9

Lesson 44.4 — Formal specifications, replay and reliability-oriented engine design 11

Module 44 project: one engine-system depth experiment 13

Primary reading directory 14

Module 44 — Advanced engine systems and simulation research

Prerequisites: Gates F-H; Modules 41-43.

Original project brief: Conduct an engine-system experiment with a formal model or numerical argument, a reference implementation and stress-tested integration. Choose one main depth area: world streaming, simulation, distributed state or reliability.

Core reference IDs: R20, R26, R54, R55, R50. Full entries are in the reading directory.

- 44.1 — Large-world scheduling, streaming and critical-path control
- 44.2 — Deformable simulation, variational models and solver tradeoffs
- 44.3 — Distributed game state, consistency and overload behavior
- 44.4 — Formal specifications, replay and reliability-oriented engine design

Lesson 44.1 — Large-world scheduling, streaming and critical-path control

Learning outcome: Design a bounded system under changing content and workload.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Large-world engines coordinate visibility, simulation, streaming and resource residency under changing demand. A frame graph can make dependencies explicit, but hidden shared state or unpredictable tasks can still expand the critical path. Partition work by ownership and locality while controlling cross-partition communication. World-coordinate precision can degrade far from the origin; origin rebasing, hierarchical coordinates or higher precision each affect physics, rendering and networking interfaces. Streaming priorities should reflect gameplay deadlines and resource availability, not only distance. Build a workload model with bursts, cancellation and scarce memory. Evaluate tail latency and quality degradation under stress rather than only steady travel through a preloaded scene.

Treat streaming as admission control, not a list of loads

A request consumes more than resident asset memory. Its compressed input, decompression scratch, decoded output, upload staging and temporary old version may overlap in lifetime. Define separate budgets and the ownership of each reservation. Admission should account for the peak overlap, not only the final asset size. A bounded queue prevents unbounded metadata growth; a resident-byte limit does not by itself bound pending allocations.

Assign a deadline, priority, estimated cost and cancellation identity to each request. Distance may predict demand during ordinary travel, but a teleport defeats that prediction. A policy can retain a low-detail representation, refuse low-priority work or show a placeholder while recovering. Those are deliberate quality/availability changes. Blocking the frame until every missing asset arrives is also a policy, usually with a very different latency contract.

The frame deadline follows dependencies

Represent frame tasks as a directed acyclic graph. Its critical path is the longest dependency chain, including synchronization and required data movement. Adding workers cannot remove a serial dependency. A background task can still delay the frame through a shared allocator lock, memory pressure or a mandatory join. Instrument readiness, queue wait, service, completion and consumption separately rather than timing only the task body.

An epoch identifies a world request generation. On teleport, increment the epoch and cancel obsolete work. A late completion must check its epoch before publishing a resource; cancellation cannot assume that a device operation already in flight instantly stops. Publication and resource retirement need explicit lifetime rules. The supplied model clears queued requests on epoch changes, but implementing late asynchronous completions is a required lab extension, not an already tested feature.

Large coordinates cross subsystem boundaries

In a binary floating-point format, absolute spacing grows with the magnitude of a coordinate. A single-precision value near 2^20 has spacing 2^(20-23)=0.125 in its represented units. Millimeter-level changes cannot all be represented there. A local origin or cell-plus-offset representation separates large world location from fine local motion; merely switching one render variable to double precision does not synchronize physics, networking and saved references.

Define which coordinate space every interface accepts. During rebasing, preserve object identity and transform histories consistently, including previous-frame positions used for motion vectors. Test transitions across cell boundaries and the origin shift during outstanding streaming work. Preserve a stable world-space identity for persistence; a temporary local coordinate should not become the sole database key.

Worked engineering example

Suppose the frame needs CPU tasks A=2 ms, B=3 ms after A, and C=4 ms after A, followed by D=1 ms after both B and C. Total work is 10 ms, but the ideal critical path is A+C+D=7 ms. With two workers and no other costs, 7 ms is attainable; dividing total work by two and claiming 5 ms ignores dependencies. If D additionally waits for a 5 ms upload started only after C, the path grows to 12 ms. Moving the upload to a background queue does not remove the dependency if D still needs its result.

Original worked check

A camera teleport invalidates a distance-based prefetch assumption and can create a sudden working-set change. The engine needs placeholders and a bounded recovery policy rather than an unlimited synchronous load.

Practical assignment

Implement a bounded world-streaming scheduler or large-coordinate experiment. Pass: teleport and memory-pressure tests preserve correctness, peak resources are measured and the critical path is explained with a trace.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_1.py`.

Reference starting point: code/gradlab/systems.py: StreamModel, Request

Assigned study

Original assignment: R26: world, task and streaming systems for comparison; R20: engine architecture; R54: numerical conditioning where coordinate precision is studied.

Focused expansion: R20 and R26: engine task, world and streaming architecture; R54: numerical representation. The reference is a discrete-event resource model, not asynchronous disk or GPU I/O.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a camera teleport defeat ordinary prefetching?
2. Why is origin rebasing an interface issue?
3. What distinguishes a bounded degradation policy?

Lesson 44.2 — Deformable simulation, variational models and solver tradeoffs

Learning outcome: Derive a physical or constraint model before comparing visual results.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Deformable simulation represents degrees of freedom and a model of forces, energies or constraints. Spatial discretization, boundary conditions and time integration together define the numerical problem. Finite-element or mass-spring approaches approximate different structures; a visually similar result does not prove equivalent material behavior. Implicit methods can improve stability for stiff systems while requiring nonlinear or linear solves and introducing other errors. Position-based and compliant-constraint methods offer alternative formulations with their own parameters and convergence behavior. Derive one simple energy or constraint, explain its units and study timestep/iteration sensitivity. Separate stability, accuracy and physical fidelity. A stable method can be inaccurate, and an accurate local solver can still use an inappropriate material model.

Start from energy, mass and a time discretization

For a one-dimensional spring displacement C(x)=x-rest, energy is E=0.5*k*C^2 and force is -k*C. If k has units N/m, compliance alpha=1/k has units m/N. A multi-particle constraint has gradients with respect to every participating position and inverse masses that govern how a correction is distributed. A pinned particle has inverse mass zero; it must not move under the constraint correction.

For implicit Euler on a linear spring system with mass matrix M, stiffness K and step h, eliminating velocity gives (M+h^2*K)*x_next=M*(x+*hv)+h^2*f, with rest offsets incorporated consistently in the force or displacement formulation. This is a numerical approximation to the dynamics, not the exact continuous trajectory. An independently assembled dense solution is a useful oracle for a small sparse/iterative implementation.

Derive compliant projection and preserve the multiplier

XPBD introduces scaled compliance alpha_tilde=alpha/h^2. For one constraint, the multiplier increment is delta_lambda=(-C-alpha_tilde*lambda)/(sum_i w_i*||grad_i C||^2+alpha_tilde). Positions receive delta_x_i=w_i*grad_i C*delta_lambda. Accumulate lambda across solver iterations within the step; resetting it at every iteration changes the method. The simple reference resets it at each new timestep and retains it across that timestep's iterations. [S05]

For a scalar unit-mass constraint with alpha=0.01, h=0.1 and predicted displacement 1, alpha_tilde=1. Starting lambda=0 gives delta_lambda=-1/(1+1)=-0.5, so the corrected displacement is 0.5. The accumulated lambda is -0.5 and the constraint-plus-compliance residual is 0.5+1*(-0.5)=0. An extra identical iteration makes no change. This is compliant equilibrium for the step, not a failed attempt to enforce C=0.

Separate model, discretization and solve error

A simulation has at least three distinct error sources: the chosen physical model, time/space discretization and incomplete numerical solution. Changing iterations primarily changes solve error. Changing h changes the discrete time problem as well as alpha_tilde. Changing mesh spacing without rescaling material parameters can change the represented physical material. Vary these independently and compare at equal simulated time.

The executable bar is a pinned one-dimensional chain of unit-mass displacements with linear neighboring springs. Its converged XPBD result can be compared with a separately assembled implicit-Euler solve of the same discrete problem. It is not a three-dimensional cloth implementation, collision solver or general nonlinear finite-element model. Use its agreement as a stepping stone to a bounded paper result; do not generalize it to all deformable simulation. Stable pictures alone do not establish accurate forces, constraint errors or material behavior.

Worked engineering example

For the scalar values above, resetting lambda to zero on each iteration produces x=0.5, then 0.25, then 0.125. The solver incorrectly drives a compliant spring toward a hard constraint as iterations increase. Keeping lambda gives x=0.5 after the first iteration and thereafter. This one-variable counterexample isolates the role of multiplier accumulation more clearly than a cloth screenshot. Use the reset-every-iteration variant as an explicitly incorrect ablation, not as an alternative implementation of XPBD.

Original worked check

For spring energy E=0.5*k*(x-rest)^2, the force is -dE/dx=-k*(x-rest). This derivative supplies a check before extending to a multi-particle system.

Practical assignment

Implement a small cloth, elastic bar or compliant-constraint model and compare two integration/solver choices. Pass: derive the core equations, check gradients, quantify constraint or energy error and reproduce one bounded result from the assigned research.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_2.py`.

Reference starting point: code/gradlab/systems.py: xpbd_scalar, bar_step, simulate_bar; code/run_experiments.py: bar

Assigned study

Original assignment: R54: equilibrium, variational and differential-equation methods; R55: 2016 XPBD and related position-based simulation course notes.

Focused expansion: S05 / R55: Macklin, Mueller and Chentanez, XPBD (2016), equations 10-18; R54: variational mechanics and linear systems. Reproduce the selected result before calling an extension research evidence.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What force follows from the example spring energy?
2. Does numerical stability guarantee physical accuracy?
3. Why vary timestep and solver iterations independently?

Lesson 44.3 — Distributed game state, consistency and overload behavior

Learning outcome: Specify which guarantees survive network delay and partial failure.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Distributed engine services divide state across processes or machines. Some operations require a single authoritative order, while others tolerate delayed or approximate views. Consistency is a property of a specified operation and model, not a single adjective that applies equally to every subsystem. During communication failure, availability and agreement requirements can conflict. Partition ownership, migration and recovery need explicit protocols. A world handoff must not create two authorities granting the same irreversible outcome. Queues require bounds and backpressure; retry storms can magnify overload. Test fault sequences involving delayed messages, crashes and duplicate requests. Use local process simulation before public infrastructure, and never claim internet-scale behavior from a two-process happy-path demo.

Specify the operation that requires authority

A position snapshot may tolerate delay while granting a unique item requires an unambiguous committed decision. Define operations, their observable effects and the consistency requirement of each. A server believing it owns an entity is not sufficient when old messages can arrive after ownership changes. The receiver that commits an irreversible effect must enforce the ownership rule.

A fencing token is a monotonically advancing ownership epoch checked by that committing receiver. After epoch 8 replaces epoch 7, an epoch-7 request must be rejected even if its sender still believes its lease is valid. The transfer of ownership itself needs a correctly coordinated, durable protocol in a real distributed system. The teaching model assumes a single atomic store that updates authority and checks commits; it does not implement replicated consensus or survive an actual machine power loss. [S11, S17]

Make duplicate handling part of the transaction

An accepted operation needs an idempotency identity and a recorded result. If a client times out after the server commits, its retry must not repeat the effect. Check the request ID, validate its payload consistency, apply the effect and record its result atomically relative to competing operations. A repeated ID with a different payload is a conflict, not a free new request.

Deduplication storage is finite. Expiring an ID without a declared retry horizon can make a very late duplicate appear new. Keeping every ID forever is an unbounded resource promise. State a retention/epoch policy or reject new work when the bounded educational store fills. Exactly-once effects require a specified boundary and failure model; a transport protocol's delivery behavior does not by itself guarantee the application's transaction semantics.

Test histories, not just individual error codes

A useful fault schedule is: old owner sends a request, the request is delayed, ownership transfers, new owner commits, the delayed old request arrives, then a retry is duplicated. Check the complete committed history and invariant, not only whether each function returned a plausible status. Introduce crashes before and after the imagined commit boundary to expose what must be persisted.

Under overload, bound ingress queues, outstanding requests and retries. Backoff with jitter can reduce synchronization, but a retry budget and deadline are still needed to stop unlimited amplification. Distinguish rejection, timeout and unknown commit outcome. Test the recovery path when the deduplication capacity is exhausted. Local deterministic schedules are excellent semantic tests; they are not measurements of internet latency or evidence of global service reliability.

Worked engineering example

Owner A holds epoch 7. Its award request is delayed. Authority changes atomically to B at epoch 8; B commits award request R. When A's epoch-7 operation reaches the authoritative sink, the sink rejects it before applying an effect. A retry of B's same request R returns its recorded result without re-awarding. Without the sink's epoch check, a correct-looking handoff acknowledgment cannot stop A's delayed request from committing. Without atomic deduplication, two simultaneous copies of R could both pass a separate check and award twice.

Original worked check

If two servers both accept ownership of the same entity during a failed handoff, both might award a unique item. A versioned ownership or fencing mechanism must prevent stale authority from committing that effect.

Practical assignment

Model a two-partition entity handoff with injected failure. Pass: a stated safety invariant holds, recovery assumptions are explicit and overload is bounded without unbounded retry amplification.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_3.py`.

Reference starting point: code/gradlab/systems.py: FencedStore

Assigned study

Original assignment: R17-R18: transport and transactions; R20: game-state architecture; use Module 41's claim and experiment discipline for the distributed model.

Focused expansion: S11: Ongaro and Ousterhout, Raft, replicated state-machine assumptions; S17: Consul session and locking semantics. These sources distinguish coordinated authority from a local locking intuition; the supplied store is not a consensus implementation.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why is duplicate authority dangerous?
2. What is a retry storm?
3. Can a small local demo establish internet-scale reliability?

Lesson 44.4 — Formal specifications, replay and reliability-oriented engine design

Learning outcome: Use precise invariants to reduce the space of subtle failures.

Planning: 28-44 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A formal model abstracts a system to states and transitions so properties can be examined precisely. Safety states that something bad never happens; liveness states that something good eventually happens under assumptions. An engine resource manager can have safety invariants about ownership and liveness obligations about eventual completion or cancellation. Model checking explores the chosen abstraction, not every detail of the production implementation. Connect model transitions to real code and tests. Deterministic replay, fault injection and property-based testing complement formal reasoning by exercising implemented behavior. Avoid vacuous proofs from impossible environment assumptions. A reliability claim should identify the property, model, bound and implementation evidence separately.

Define states and transitions before selecting a prover

A formal specification starts with variables, allowed initial states and a transition relation. For an asset request, useful state includes owner-alive, request-issued, request-pending and whether an invalid callback occurred. State the invariant: a callback must not touch an owner after release. A transition that completes pending work must check owner validity or use a lifetime mechanism that guarantees the target still exists.

Safety says that forbidden behavior never occurs. A finite bad prefix can demonstrate a safety violation. Liveness instead demands eventual progress, such as each accepted request eventually completing or being canceled. A model that allows the worker to be ignored forever cannot promise completion without a scheduling or fairness assumption. Fairness is a substantive environment requirement, not a magic word added after the proof. [S12]

A finite model can be complete for itself and incomplete for the engine

Breadth-first search of all reachable states is exhaustive for a finite model when its transition relation and initial states are fully represented. Store predecessor edges to recover a minimal-step counterexample. The supplied one-request model explores this graph and deliberately disables the callback guard to expose the sequence request, release, complete. Passing the guarded model says nothing by itself about arbitrary request counts, memory reclamation, real pointers or device callbacks.

Connect each model transition to code and an implementation test. Check that assumptions do not eliminate the very failures you intend to study. For example, declaring release impossible while work is pending would make the ownership test vacuous if real callers can release then. Mutation testing helps: a deliberately missing guard must produce a counterexample, proving that the checked property can detect the intended class of mistake.

Replay preserves evidence only when nondeterminism is captured

A seed alone is not always a replay record. Input order, clock reads, external responses, thread scheduling, floating-point environment and random-number consumption can affect results. Define which nondeterminism is recorded, controlled or excluded. Hash canonical serialized states at checkpoints and compare the first divergence, but retain the underlying state or event log so a mismatch is diagnosable.

The reference hash sorts JSON object keys and uses a stable digest; it is not a cross-platform floating-point determinism guarantee or a cryptographic proof of correct simulation. Replay and fault injection complement model checking by exercising actual code. Keep the evidence labels separate: model safety, conditional liveness argument, implementation tests and measured reliability are different claims.

Worked engineering example

Start with alive=true and no pending request. The request transition sets pending=true. Release sets alive=false. In the buggy model, complete unconditionally dispatches the callback, setting bad=true. The resulting three-transition trace is a safety counterexample. In the guarded model, complete clears pending but does not call the dead owner. This fixes the stated safety property. It does not prove liveness: a scheduler that never executes complete or cancel can leave pending=true forever. A progress assumption and separate liveness analysis are required.

Original worked check

For an asset request, safety might require that a released owner never receives a callback touching its memory. Liveness might require that each accepted request eventually completes or cancels, assuming the worker continues to run.

Practical assignment

Model a small engine lifecycle or queue protocol and connect it to implementation tests. Pass: at least one safety and one conditional liveness property are defined, a deliberate violation is found and abstraction limits are documented.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_4.py`.

Reference starting point: code/gradlab/systems.py: lifecycle_check, replay_hash

Assigned study

Original assignment: R50: safety, liveness and assumptions as a formal-methods reference; R11 and Module 18: ownership/concurrency contracts; R26: system comparison.

Focused expansion: S12: Lamport, Safety, Liveness, and Fairness; R50: formal assumptions and bounded verification. The included exhaustive search establishes safety only for its small explicit model.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How do safety and liveness differ?
2. Why must liveness assumptions mention scheduling or worker progress?
3. Does proving an abstract model automatically prove all implementation code?

Module 44 project: one engine-system depth experiment

Choose world streaming, deformable simulation, distributed state or lifecycle reliability as the main depth area. Complete all four lesson labs at their bounded scale, then integrate one area more deeply. Write a numerical argument or state/transition model before comparing implementations.

Streaming evidence must include teleport, late completion, transient memory and a bounded degradation policy. Simulation evidence must separate model, discretization and solve error with timestep/iteration sweeps. Distributed-state evidence must identify the authoritative sink, deduplication/commit boundary and persistence assumptions under fault histories. Reliability evidence must distinguish checked safety from conditional liveness and connect the model to implemented tests.

Required artifacts: contract, reference model, implementation, deliberately failing case, stress trace, parameter sweep and limitations. Model/argument 30 points, implementation correctness 30, failure/overload evaluation 25, reproducibility 15. Suggested pass: 80/100 with no unbounded resource promise or unqualified reliability claim.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R11 — C++ Core Guidelines

C++ Core Guidelines editors

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R17 — RFC 9293: Transmission Control Protocol

IETF / RFC Editor

https://www.rfc-editor.org/rfc/rfc9293.html

Transport semantics; reference reading after networking foundations.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R18 — PostgreSQL tutorial

PostgreSQL Global Development Group

https://www.postgresql.org/docs/current/tutorial.html

Relational data, transactions and SQL; pin local server and client versions.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University

https://graphics.cs.cmu.edu/courses/15-466/

Runtime systems and game-production programming.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R26 — Programming with C++ in Unreal Engine

Epic Games

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

Engine comparison after C++ foundations, not a beginner substitute.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R50 — SymbiYosys documentation

YosysHQ

https://yosyshq.readthedocs.io/projects/sby/en/latest/

Formal hardware verification, safety, liveness and cover properties.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R54 — 18.085 Computational Science and Engineering I

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R55 — Author publication archive: position-based simulation and XPBD

Matthias Mueller and collaborators

https://matthias-research.github.io/pages/publications/publications.html

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S05 — XPBD: Position-Based Simulation of Compliant Constrained Dynamics (2016)

Miles Macklin, Matthias Mueller and Nuttapong Chentanez

https://matthias-research.github.io/pages/publications/XPBD.pdf

Original paper; multiplier/compliance equations inspected. A bounded reproduction target, not a completed result supplied here.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S11 — In Search of an Understandable Consensus Algorithm, extended version

Diego Ongaro and John Ousterhout

https://raft.github.io/raft.pdf

Replicated state-machine assumptions; PDF and architecture page inspected. The teaching sink does not implement Raft.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S12 — Safety, Liveness, and Fairness (2019)

Leslie Lamport

https://lamport.azurewebsites.net/tla/safety-liveness.pdf

Precise definitions and fairness assumptions; safety/liveness page inspected.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S17 — Consul session semantics

HashiCorp

https://developer.hashicorp.com/consul/docs/automate/session

Session/locking behavior and authority limitations; primary service documentation, not a service deployment requirement.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.